JavaScript is disabled on your browser.

compact1, compact2, compact3

java.io

Class ByteArrayOutputStream

java.lang.Object
    java.io.OutputStream
        java.io.ByteArrayOutputStream

All Implemented Interfaces:

`Closeable`, `Flushable`, `AutoCloseable`

```
public class ByteArrayOutputStream
extends OutputStream
```

This class implements an output stream in which the data is written into a byte array. The buffer automatically grows as data is written to it. The data can be retrieved using `toByteArray()` and `toString()`.

Closing a `ByteArrayOutputStream` has no effect. The methods in this class can be called after the stream has been closed without generating an `IOException`.

Since:

`JDK1.0`

Field Summary

Fields

Modifier and Type	Field	Description
`protected byte[]`	**`buf`**	The buffer where data is stored.
`protected int`	**`count`**	The number of valid bytes in the buffer.

Constructor Summary

Constructors

Constructor	Description
`ByteArrayOutputStream()`	Creates a new byte array output stream.
`ByteArrayOutputStream(int size)`	Creates a new byte array output stream, with a buffer capacity of the specified size, in bytes.

Method Summary

Concrete Methods

Deprecated Methods

Modifier and Type	Method	Description
`void`	`close()`	Closing a `ByteArrayOutputStream` has no effect.
`void`	`reset()`	Resets the `count` field of this byte array output stream to zero, so that all currently accumulated output in the output stream is discarded.
`int`	`size()`	Returns the current size of the buffer.
`byte[]`	`toByteArray()`	Creates a newly allocated byte array.
`String`	`toString()`	Converts the buffer's contents into a string decoding bytes using the platform's default character set.
`String`	`toString(int hibyte)`	**Deprecated.** This method does not properly convert bytes into characters. As of

		JDK 1.1, the preferred way to do this is via the `toString(String enc)` method, which takes an encoding-name argument, or the `toString()` method, which uses the platform's default character encoding.
String	**toString**(**String** charsetName)	Converts the buffer's contents into a string by decoding the bytes using the named **charset**.
void	**write**(byte[] b, int off, int len)	Writes `len` bytes from the specified byte array starting at offset `off` to this byte array output stream.
void	**write**(int b)	Writes the specified byte to this byte array output stream.
void	**writeTo**(**OutputStream** out)	Writes the complete contents of this byte array output stream to the specified output stream argument, as if by calling the output stream's write method using `out.write(buf, 0, count)`.

Methods inherited from class java.io.OutputStream

`flush`, `write`

Methods inherited from class java.lang.Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Field Detail

buf

```
protected byte[] buf
```

The buffer where data is stored.

count

```
protected int count
```

The number of valid bytes in the buffer.

Constructor Detail

ByteArrayOutputStream

```
public ByteArrayOutputStream()
```

Creates a new byte array output stream. The buffer capacity is initially 32 bytes, though its size increases if necessary.

ByteArrayOutputStream

```
public ByteArrayOutputStream(int size)
```

Creates a new byte array output stream, with a buffer capacity of the specified size, in bytes.

Parameters:

`size - the initial size.`

Throws:

`IllegalArgumentException - if size is negative.`

Method Detail

write

```
public void write(int b)
```

Writes the specified byte to this byte array output stream.

Specified by:

`write` in class `OutputStream`

Parameters:

`b` - the byte to be written.

write

```
public void write(byte[] b,
                  int off,
                  int len)
```

Writes `len` bytes from the specified byte array starting at offset `off` to this byte array output stream.

Overrides:

`write` in class `OutputStream`

Parameters:

`b` - the data.

`off` - the start offset in the data.

`len` - the number of bytes to write.

writeTo

```
public void writeTo(OutputStream out)
             throws IOException
```

Writes the complete contents of this byte array output stream to the specified output stream argument, as if by calling the output stream's write method using `out.write(buf, 0, count)`.

Parameters:

`out` - the output stream to which to write the data.

Throws:

`IOException` - if an I/O error occurs.

reset

```
public void reset()
```

Resets the `count` field of this byte array output stream to zero, so that all currently accumulated output in the output stream is discarded. The output stream can be used again, reusing the already allocated buffer space.

See Also:

`ByteArrayInputStream.count`

toByteArray

```
public byte[] toByteArray()
```

Creates a newly allocated byte array. Its size is the current size of this output stream and the valid contents of the buffer have been copied into it.

Returns:

`the current contents of this output stream, as a byte array.`

See Also:

`size()`

size

```
public int size()
```

Returns the current size of the buffer.

Returns:

`the value of the count field, which is the number of valid bytes in this output stream.`

See Also:

`count`

toString

```
public String toString()
```

Converts the buffer's contents into a string decoding bytes using the platform's default character set. The length of the new `String` is a function of the character set, and hence may not be equal to the size of the buffer.

This method always replaces malformed-input and unmappable-character sequences with the default replacement string for the platform's default character set. The CharsetDecoder class should be used when more control over the decoding process is required.

Overrides:

`toString` in class `Object`

Returns:

`String decoded from the buffer's contents.`

Since:

JDK1.1

toString

```
public String toString(String charsetName)
                throws UnsupportedEncodingException
```

Converts the buffer's contents into a string by decoding the bytes using the named `charset`. The length of the new `String` is a function of the charset, and hence may not be equal to the length of the byte array.

This method always replaces malformed-input and unmappable-character sequences with this charset's default replacement string. The `CharsetDecoder` class should be used when more control over the decoding process is required.

Parameters:

`charsetName - the name of a supported charset`

Returns:

`String decoded from the buffer's contents.`

Throws:

`UnsupportedEncodingException - If the named charset is not supported`

Since:

`JDK1.1`

toString

```
@Deprecated
public String toString(int hibyte)
```

Deprecated. *This method does not properly convert bytes into characters. As of JDK 1.1, the preferred way to do this is via the `toString(String enc)` method, which takes an encoding-name argument, or the `toString()` method, which uses the platform's default character encoding.*

Creates a newly allocated string. Its size is the current size of the output stream and the valid contents of the buffer have been copied into it. Each character *c* in the resulting string is constructed from the corresponding element *b* in the byte array such that:

```
c == (char)(((hibyte & 0xff) << 8) | (b & 0xff))
```

Parameters:

hibyte - the high byte of each resulting Unicode character.

Returns:

the current contents of the output stream, as a string.

See Also:

size(), toString(String), toString()

close

```
public void close()
           throws IOException
```

Closing a ByteArrayOutputStream has no effect. The methods in this class can be called after the stream has been closed without generating an IOException.

Specified by:

close in interface Closeable

Specified by:

close in interface AutoCloseable

Overrides:

close in class OutputStream

Throws:

IOException - if an I/O error occurs.

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.